XML in PHP 5

ApacheCon

November 17, 2003. Sin City, Baby!

Adam Trachtenberg <adam@trachtenberg.com>

Who Am I?

Who am I?

- Programming in PHP since php/fi
- Co-author PHP Cookbook (O'Reilly 2003)
- Author PHP 5 Essentials (O'Reilly 2004)

Intro

What's New XMLwise in PHP5?

- Completely Rewritten
- libxml2 & libxsl Based
- Standards Compliant
- SimpleXML

Topics

What's Will You Learn Today?

- SimpleXML
- -- Amazon REST
- -- Before SAX
- -- After SimpleXML
- -- XPath Query
- DOM
- -- RSS 0.9 and 1.0
- -- Create
- -- Read
- -- Read w/ Namespaces
- -- XPath Query
- -- XSLT Transform

SimpleXML

- Easiest way to use XML.
- foreach your way through the document.
- Better than 80/20. It's 95/5.
- I <heart> SimpleXML.

Amazon.Com & REST

- Query Amazon.Com for book information using REST.
- Is everyone familiar with REST?
- What about Amazon.Com?
- Somewhat real life and somewhat complex.

Before: REST & SAX

```
#!/usr/local/php
<?php
$base ='http://xml.amazon.com/onca/xml3';
$query_string = '';

$params = array( 'ManufacturerSearch' => "O'Reilly",
                 'mode'               => 'books',
                 'sort'               => '+salesrank',
                 'page'               => 1,
                 'type'               => 'lite',
                 'f'                  => 'xml',
                 't'                  => 'trachtenberg-20',
                 'dev-t'              => 'XXXXX',
               );

foreach ($params as $key => $value) {
   $query_string .= "$key=" . urlencode($value) . "&";
}

$url = "$base?$query_string";

$c = curl_init($url);
curl_setopt($c, CURLOPT_RETURNTRANSFER, 1);
$hits = curl_exec($c);
curl_close($c);

$obj = new Amazon_parser;
$xml = xml_parser_create();
xml_set_object($xml, $obj);
xml_set_element_handler($xml, 'start_element', 'end_element');
xml_set_character_data_handler($xml, 'character_data');
xml_parser_set_option($xml, XML_OPTION_CASE_FOLDING, false);
xml_parse($xml, $hits);
xml_parser_free($xml);

class Amazon_parser {

  var $tag;
  var $item;

  function start_element($parser, $tag, $attributes) {
    // Each new item lives inside a Details container
    if ('Details' == $tag) {
      // Make a new Item
      $this->item = new Amazon_item;

      // Loop through the attributes
```

```
      foreach ($attributes as $name => $value) {
        if (isset($this->item->$name)) {
          $this->item->$name .= trim($value);
        }
      }
    } elseif (!empty($this->item)) {
      $this->tag = $tag;
    }
  }

  function end_element($parser, $tag) {
    // Each product ends with a closing Details tag
    if ('Details' == $tag) {
      $this->item->display();
      unset($this->item);
    }
  }

  function character_data($parser, $data) {
    if (!empty($this->item) and $data = trim($data)) {
      // only add pre-declared character data elements
      if (isset($this->item->{$this->tag})) {
        if(is_array($this->item->{$this->tag})) {
          $this->item->{$this->tag}[] = $data;
        } else {
          $this->item->{$this->tag}.= $data;
        }
      }
    }
  }
}

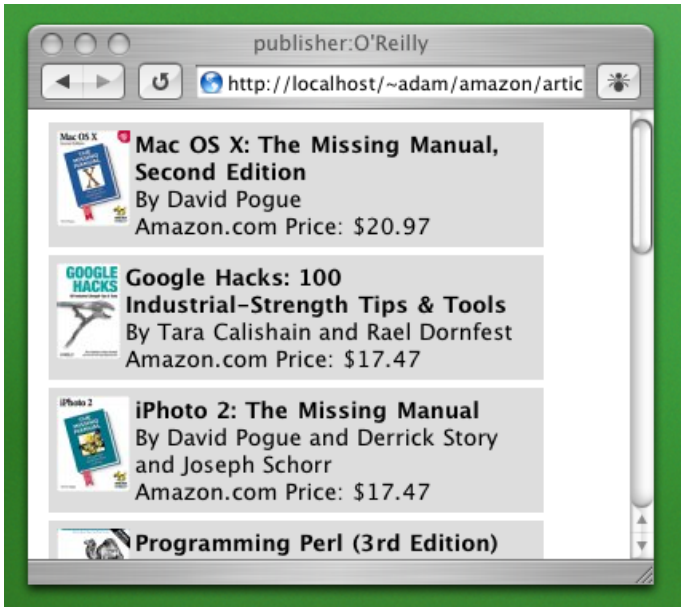
class Amazon_item {

  // attributes and character data fields we want
  var $ProductName = '' ;
  var $Author = array();
  var $OurPrice = '';
  var $url = '';
  var $ImageUrlSmall = '';

  function display() {
    printf('<div style="clear:left; width: 300px; padding:5px;
            margin:5px; background:#ddd;">
            <a href="s"><image src="s"
            alt="%s" align="left"/></a>
            <b>%s</b><br/>
            %s<br/>
            Amazon.com Price: %s<br/>
            </div>',
```

```
            $this->url, $this->ImageUrlSmall,
            htmlspecialchars($this->ProductName),
            htmlspecialchars($this->ProductName),
            htmlspecialchars(join(' and ', $this->Author)),
            htmlspecialchars($this->OurPrice)
          );
  }
}
?>
```

Before: REST & SAX: Results



```
<?php
$base ='http://xml.amazon.com/onca/xml3';
$query_string = '';

$params = array( 'ManufacturerSearch' => "O'Reilly",
                 'mode'               => 'books',
                 'sort'               => '+salesrank',
                 'page'               => 1,
                 'type'               => 'lite',
                 'f'                  => 'xml',
                 't'                  => 'trachtenberg-20' ,
                 'dev-t'              => 'XXXXX' ,
               );

foreach ($params as $key => $value) {
   $query_string .= "$key=" . urlencode($value) . "&";
}

$url = "$base?$query_string";

$hits = simplexml_load_file($url);

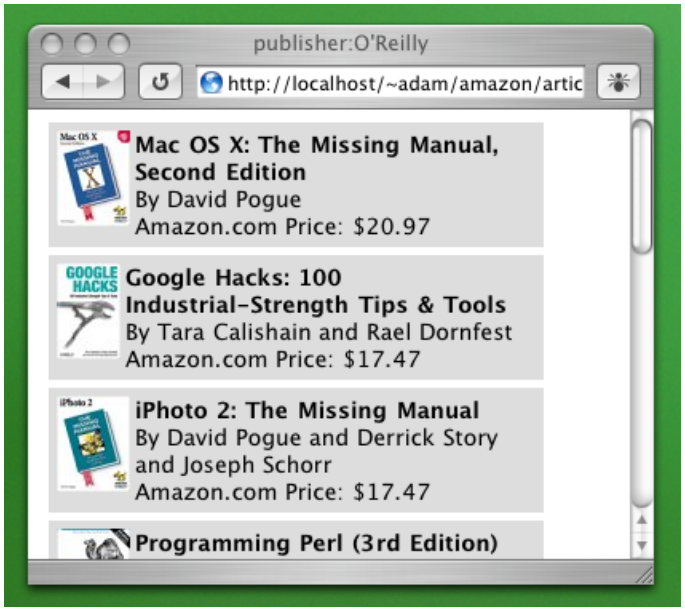
foreach($hits->Details as $hit) {
    $a = $hit->Authors->Author;
    $a = (is_array($a)) ? join(' and ', $a) : $a;

    printf('<div style="clear:left; width: 300px; padding:5px; margin:5px; background:#ddd;">
            <a href="s"><image src="s" alt="s" align="left"/></a><b>s</b><br/>
            %s<br/>
            Amazon.com Price: %s<br/></div>',

            $hit->url, $hit->ImageUrlSmall, $hit->ProductName, $hit->ProductName,
            join(' and ', $a),
            $hit->OurPrice
          );
?>
```

REST and SimpleXML

After: REST & SimpleXML: Results



SimpleXML also supports XPath

```
<?php
$s = simplexml_load_file('rss0.9.xml');

// find and print all <title> elements
foreach($s->xsearch('//title') as $t) { print "$t\n"; }

// find and print every other <title> element
// living below an <item subject="PHP"> element
foreach($s->xsearch('descendant::item[attribute::subject="PHP"]/
          title[position() mod 2 = 1]') as $t) {
   print "$t\n";
}
?>
```

DOM

DOM is the mother of all XML extensions

- W3 Standard.
- Platform- and Language-Neutral.
- Completely comprehensive.
- And a pain in the ass.

DOM Details

DOM lets you create / edit / append / delete:

- Documents: One of those XML things
- Document Fragments: Almost XML
- Elements: <tag>
- Attributes: %attribute="value"
- Text: Stuff inside elements
- Comments: <!-- Comment -->
- Processing Instructions: <php... ?>
- CDATA: <![CDATA[...]]>
- Entities: &entity;

Creating XML elements using DOM

Adding a new RSS item to an existing feed

```
<?php
$dom = domDocument::load('rss.xml');
$item = $dom->documentElement->appendChild(new domElement('item'));

$title = $item->appendChild(new domElement('title', 'PHP.net'));
$link = $item->appendChild(new domElement('link', 'http://www.php.net/'));
$description = $item->appendChild(new domElement('description', 'PHP Home Page'));

$dom->save('rss.xml');
?>
```

Reading XML w/DOM


```
<?php
$dom = domDocument::load('rss0.9.xml');
$items = $dom->getElementsByTagname('item');

foreach ($items as $item) {
        $title = array_pop($item->getElementsByTagname('title'));
        $title_text_value = $title->firstChild->nodeValue;

        $link = array_pop($item->getElementsByTagname('link'));
        $link_text_value = $link->firstChild->nodeValue;

        $description = array_pop($item->getElementsByTagname('description'));
        $description_text_value = $description->firstChild->nodeValue;

        print "<p><a href=\"$link_text_value\">$title_text_value</a><br />\n";
        print "$description_text_value</p>\n";
}
?>
```

Reading Namespaced XML w/DOM


```
<?php
$dom = domDocument::load('rss1.0.xml');

$rss = 'http://purl.org/rss/1.0/';
$items = $dom->getElementsByTagNameNS($rss, 'item');

foreach ($items as $item) {
        $title = array_pop($item->getElementsByTagnameNS($rss, 'title'));
        $title_text_value = $title->firstChild->nodeValue;

        $link = array_pop($item->getElementsByTagnameNS($rss, 'link'));
        $link_text_value = $link->firstChild->nodeValue;

        $description = array_pop($item->getElementsByTagnameNS($rss, 'description'));
        $description_text_value = $description->firstChild->nodeValue;

        print "<p><a href=\"$link_text_value\">$title_text_value</a><br />\n";
        print "$description_text_value</p>\n";
}
?>
```

```
<?php
$dom = domDocument::load('rss1.0.xml');

$xpath = new domXPath($dom);
$xpath->register_namespace('rss', 'http://purl.org/rss/1.0/');
$rss = 'http://purl.org/rss/1.0/';

$items = $xpath->query('//rss:item');

foreach ($items as $item) {
        $title = array_pop($item->getElementsByTagnameNS($rss, 'title'));
        $title_text_value = $title->firstChild->nodeValue;

        $link = array_pop($item->getElementsByTagnameNS($rss, 'link'));
        $link_text_value = $link->firstChild->nodeValue;

        $description = array_pop($item->getElementsByTagnameNS($rss, 'description'));
        $description_text_value = $description->firstChild->nodeValue;

        print "<p><a href=\"$link_text_value\">$title_text_value</a><br />\n";
        print "$description_text_value</p>\n";
}
?>
```

DOM and XSLT

DOM also works with XSLT

```
<?php
// Query Amazon
$xml = domDocument::load($url);

// Load local XSL stylesheet
$xsl = domDocument::load('amazon.xsl');

// Make XSLT object; load in stylesheet
$xslt = new xsltprocessor();
$xslt->importStylesheet($xsl);

// Transform!
print $xslt->transformToXML($xml);
?>
```

Resources

For More Information

- DOM Spec: http://www.w3.org/DOM/
- XPath: http://www.w3.org/TR/xpath
- XSL(T?): http://www.w3.org/Style/XSL/

Index

Who Am I? 2
Intro 3
Topics 4
SimpleXML 5
SimpleXML in Action 6
REST and SAX 7
REST and SAX 10
REST and SimpleXML 11
REST and SimpleXML 12
XPath 13
DOM 14
DOM Details 15
Creating XML w/DOM 16
Reading XML w/DOM 17
Reading Namespaced XML w/DOM 18
DOM and XPath 19
DOM and XSLT 20
Resources 21

This presenation has ended.
Click anywhere to return.

_

